Detecting Refactoring Commits in Machine Learning Python Projects: A Machine Learning-Based Approach

SHAYAN NOEI, Electrical and Computer Engineering, Queen's University, Kingston, Ontario, Canada
HENG LI, School of Computing, Queen's University, Kingston, Ontario, Canada
YING ZOU, Queen's University, Kingston, Ontario, Canada

Refactoring aims to improve the quality of software without altering its functional behaviors. Understanding developers' refactoring activities is essential to improve software maintainability. The use of machine learning (ML) libraries and frameworks in software systems has significantly increased in recent years, making the maximization of their maintainability crucial. Due to the data-driven nature of ML libraries and frameworks, they often undergo a different development process compared to traditional projects. As a result, they may experience various types of refactoring, such as those related to the data. The state-of-the-art refactoring detection tools have not been tested in the ML technical domain, and they are not specifically designed to detect ML-specific refactoring types (e.g., data manipulation) in ML projects; therefore, they may not adequately find all potential refactoring operations, specifically the ML-specific refactoring operations. Furthermore, a vast number of ML libraries and frameworks are written in Python, which has limited tooling support for refactoring detection. PyRef, a rule-based and state-of-the-art tool for Python refactoring detection, can identify 11 types of refactoring operations with relatively high precision. In contrast, for other languages such as Java, state-of-the-art tools are capable of detecting a much more comprehensive list of refactorings. For example, Rminer can detect 99 types of refactoring for Java projects. Inspired by previous work that leverages commit messages to detect refactoring, we introduce MLRefScanner, a prototype tool that applies ML techniques to detect refactoring commits in ML Python projects. MLRefScanner detects commits involving both ML-specific refactoring operations and additional refactoring operations beyond the scope of state-of-the-art refactoring detection tools. To demonstrate the effectiveness of our approach, we evaluate MLRefScanner on 199 ML open source libraries and frameworks and compare MLRefScanner against other refactoring detection tools for Python projects. Our findings show that MLRefScanner outperforms existing tools in detecting refactoring-related commits, achieving an overall precision of 94% and recall of 82% for identifying refactoring-related commits. MLRefScanner can identify commits with ML-specific and additional refactoring operations compared to state-of-the-art refactoring detection tools. When combining MLRefScanner with PyRef, we can further increase the precision and recall to 95% and 99%, respectively. MLRefScanner provides a valuable contribution to the Python ML community, as it allows ML developers to detect refactoring-related commits more effectively in their ML Python projects. Our study sheds light on the promising direction of leveraging machine learning techniques to detect refactoring activities for other programming languages or technical domains where the commonly used rule-based refactoring detection approaches are not sufficient.

CCS Concepts: • **Software and its engineering → Maintaining software**;

Authors' Contact Information: Shayan Noei (corresponding author), Electrical and Computer Engineering, Queen's University, Kingston, Ontario, Canada; e-mail: s.noei@queensu.ca; Heng Li, School of Computing, Queen's University, Kingston, Ontario, Canada; e-mail: heng.li@polymtl.ca; Ying Zou, Queen's University, Kingston, Ontario, Canada; e-mail: ying.zou@queensu.ca.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2025/2-ART68
https://doi.org/10.1145/3705309

Additional Key Words and Phrases: Code Refactoring, Refactoring Detection, Python Refactoring, Machine Learning, Code Quality

ACM Reference format:
Shayan Noei, Heng Li, and Ying Zou. 2025. Detecting Refactoring Commits in Machine Learning Python Projects: A Machine Learning-Based Approach. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 68 (February 2025), 25 pages.
https://doi.org/10.1145/3705309

1 Introduction

Understanding the appropriate timing and implementation of refactoring in software development enables developers to comprehend software evolution, make informative decisions, and enhance their knowledge of code design [4, 9]. However, it is challenging to gain insights into maintenance practices, as developers often provide insufficient documentation regarding their refactoring activities. Quite often, developers use general terms (e.g., clean-up) to describe the refactoring actions instead of providing specific details [4].

Sophisticated tools (e.g., Rminer [71, 72]) are available for other programming languages, such as Java. However, refactoring detection tools for Python are relatively limited. Moreover, existing refactoring detection tools are not specifically designed or tested to identify domain-specific (e.g., **Machine Learning (ML)**) refactorings. The existing refactoring detection tools primarily rely on pattern-matching using **Abstract Syntax Tree (AST)** [9, 71, 72] to detect refactoring operations. Consequently, they are unable to identify accessory information in the code, such as logical refactorings that do not follow a specific pattern. These refactorings are comprehended by developers but cannot be identified by pattern-matching in the AST structure changes.

Python has become one of the most popular programming languages, especially for developing AI and ML libraries, frameworks, and applications [35, 45, 58, 68]. The utilization of ML frameworks and libraries has increased in recent years, and many software projects now depend on ML libraries and frameworks [18, 28]. Therefore, ensuring the proper maintainability of the ML libraries and frameworks is essential. Refactoring is a technique used to improve the maintainability of software [2, 7, 53]. Due to the data-driven nature of ML projects, they undergo a different development process, including design and implementation, which is more complex than traditional software development [6, 15, 74]. Various refactoring experiences and types may exist in ML-specific contexts, e.g., data modifications like switching dataset types (e.g., from CSV to Parquet) or altering the way of iterating through dataset records.

Compared to other programming languages, Python prioritizes readability [24, 37] and has a simpler coding structure [1, 37]. For instance, Python is a dynamically typed language, while languages like Java are statically typed [37]. Consequently, identifying class and object relationships through static code analysis in Python is affected. AST in Python provides less detailed information compared to languages like Java. Moreover, there are no expectations to improve Python AST building tools in the near future [19].

Existing approaches for refactoring detection can be classified into two categories: rule-based and keyword-based. The state-of-the-art keyword-based approaches [3, 17, 38, 59] involve searching through commit messages to identify refactoring commits based on a predefined set of keywords. For instance, if certain keywords, such as *refactor*, appear in a commit message, it is identified as a refactoring commit. However, not all refactoring activities are explicitly mentioned in commit messages. Therefore, keyword-based approaches may fail to detect refactoring commits that lack explicit keyword mentions in their commit messages.

PyRef [9] is the most effective and validated rule-based refactoring detection tool for Python. PyRef leverages pattern-matching within the AST using a predefined set of rules [9]. PyRef can detect up to 11 types of refactoring operations, covering 11% of the refactoring operations that Rminer can detect in Java code. PyRef achieved a precision of 89.6% and a recall of 76.1%. Additionally, PyRef has not been specifically designed or validated for the ML technical domain for refactoring detection, which may provide incomplete coverage of refactoring operations and increase the likelihood of false negatives in identifying refactoring commits.

In short, the keyword-based and rule-based approaches are often limited by the available keywords and rules in the studied projects. Such approaches may not work well when applied to new projects that use new keywords or unknown rules, or when used in domain-specific projects. As a result, the coverage of these approaches could decrease in new projects. In contrast, MLRefScanner integrates code, process, and textual features for refactoring detection—a topic not extensively studied but potentially valuable for future research in this area. The objective of our proposed ML approach is to extract and learn from the knowledge in existing projects and extend it for refactoring detection, particularly in scenarios where rule-based or keyword-based approaches are not sufficient to recognize new rules or keywords used in the commit messages in new projects.

In this article, we introduce MLRefScanner, a purpose-built prototype tool designed specifically to identify refactoring commits in the history of ML Python projects. Leveraging ML techniques and algorithms, MLRefScanner aims to improve the coverage of identifying refactoring operations in Python code and enhance the capabilities of detecting ML-specific refactorings. MLRefScanner allows practitioners and researchers to track and analyze refactoring activities with high precision and recall in the software development history of ML Python libraries and frameworks, enabling a deeper understanding of the scope and evolution of the ML codebase.

To evaluate the effectiveness of MLRefScanner, we conduct empirical studies on 199 ML libraries and frameworks. In this study, we aim to answer the following **Research Questions (RQs)**:

RQ1. *What is the performance of our approach for identifying refactoring commits in ML Python projects?*—To identify refactoring commits in the history of ML Python libraries and frameworks, we analyze their commit history and extract multiple dimensions of representative features (e.g., commit message-related features) to train MLRefScanner with different classifiers (e.g., Random Forest). Our findings demonstrate that LightGBM is the best-performing classifier for detecting refactoring commits, enabling MLRefScanner to achieve an overall precision of 94%, a recall of 82%, and an **Area under the Curve (AUC)** of 89%. MLRefScanner can identify refactoring commits with ML-specific and additional refactoring types compared to state-of-the-art tools.

RQ2. *What are the main features for explaining refactoring commits?*—To gain deeper insights into the nature of refactoring commits for practitioners and increase the transparency of the proposed approach, we apply a features evaluation approach using the number of decision splits and the **Local Interpretable Model-Agnostic Explanation (LIME)** interpreter [60], which helps us identify the most important features of refactoring commits. As a result, the top 10 features that indicate a refactoring commit include: (1) textual features: *update, remove, refactor, improve, and move*; (2) process features: *author's* ***Refactoring Contribution Ratio (RCR)****, lines deleted, lines added, code entropy*; and (3) code features: *lines of declarative code.*

RQ3. *Can we leverage commit information to complement existing keyword-based and rule-based approaches?*—We compare MLRefScanner with existing keyword- and rule-based approaches in identifying refactoring commits, demonstrating its superiority. Our approach outperforms

the existing best approach (i.e., PyRef) achieving a 22% increase in AUC. Additionally, we explore the potential benefits of utilizing **Ensemble Learning (EL)** to complement PyRef with MLRefScanner. The EL approach achieves an overall precision of 95% and a recall of 99%, significantly improving the low recall of the state-of-the-art approaches.

Our work makes the following main contributions:

- —We propose a learning-based approach and a prototype, MLRefScanner, for detecting refactoring commits in ML Python projects, achieving an overall precision of 94% and recall of 82%.
- —We propose an ensemble approach that combines our learning-based approach and a state-of-the-art rule-based refactoring detection approach (i.e., PyRef), further improving the performance of both MLRefScanner and PyRef.
- —We provide a carefully curated dataset that labels refactoring commits in ML Python projects.
- —We address the low recall issue of the existing state-of-the-art approaches by considering different types of features (i.e., textual, process, and code) in refactoring commit detection.
- —We present the first prototype of an ML-specific refactoring commit detection tool, capable of identifying commits related to ML-specific refactoring types as well as additional commits involving general (not ML-specific) refactoring types.

The replication package of the study can be accessed online [54].

Organization. The remainder of our study is organized as follows. Section 2 describes the experiment setup of this study. Section 3 presents the motivation, approaches, and results of our RQs. Section 4 explains the generalizability of our approach to other technical domains. Section 5 provides the implications of our study. Section 6 discusses the threats to the validity of our findings. Section 7 surveys related studies and compares them to our work. Finally, we conclude our article and present future research directions in Section 8.

2 Experiment Setup

This section presents the setup of our study, including our data collection and data analysis approaches.

2.1 Overview of Our Approach

We propose an ML-based approach to detect refactoring commits in ML Python projects. An overview of our study is depicted in Figure 1. We train MLRefScanner in the ML domain using 199 ML libraries and frameworks. Leveraging the commit history on the subject projects, we create a curated dataset by labeling each commit as a refactoring commit or not, based on the commit messages and refactoring analysis results obtained from the PyRef tool [9]. We manually validate our labeling approach through a validation process involving two individuals to assess its performance. Subsequently, we extract textual, process, and code features to capture the characteristics of each commit. We then partition the data into training sets and testing sets. During the training phase, we pre-process the data by applying undersampling to address issues related to an unbalanced dataset. Additionally, we conduct hyper-parameter tuning to optimize the training settings of our classifiers. We interpret the trained model and extract the most important features to provide deeper insights into the characteristics of refactoring and non-refactoring commits. Finally, we ensemble the proposed model with the state-of-the-art refactoring detection tool (i.e., PyRef) and assess whether we can enhance the refactoring detection performance by considering a variety of features in each commit.

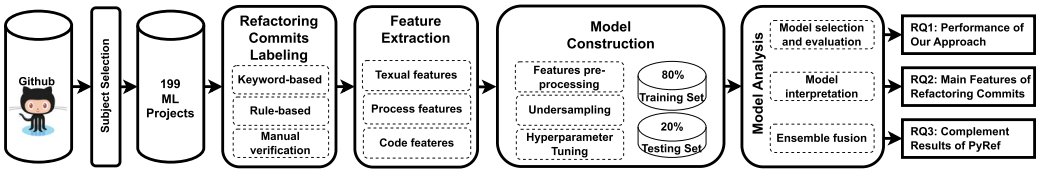


Fig. 1. Overview of the approach.

2.2 Subject Selection

We select ML projects to train and test MLRefScanner in the ML technical domain. Inspired by previous work [56], we use the advanced search on GitHub[1] to select repositories of ML libraries, frameworks, and applications primarily written in Python by specifying keywords, including *deep learning*, *reinforcement learning*, and *ML*. As a result, we identify 161,662 projects. It is time-consuming to extract refactoring commits. For example, it takes PyRef[2] approximately 1 day to process each project because it involves parsing the commit code change history and constructing an AST for each commit. We plot the distributions of project size (lines of code), stars, and forks of the repositories and adopt a systematic process [56] to select the projects. The selection criteria help us select mature and active projects that have demonstrated sustained growth and community engagement. We remove the projects that have a size smaller than 23,148 KB, fewer than 27 stars, or fewer than 18 forks as such projects are below the 3rd quartile of the distribution; have had no pushes in the year leading up to the data collection date (from 2021/04/01 to 2022/04/01); have no issues; have no downloads; are archived; are disabled; or have fewer than 1,000 commits. As a result, we obtain 217 repositories. Furthermore, we perform a manual validation to remove 18 irrelevant (i.e., non-ML repositories or repositories that contain only documentation). Finally, 199 repositories are selected as the basis (i.e., training and testing) for our experimental analysis.

2.3 Labeling Refactoring Commits

To label commits as either refactoring or non-refactoring in the selected ML projects, we employ two state-of-the-art automated approaches: (1) keyword-based labeling and (2) rule-based labeling with PyRef. If a commit is identified as a refactoring by either approach, we label it as a refactoring commit. During the labeling process, we classify 155,780 commits as refactoring and 343,187 as non-refactoring commits. We acknowledge that the automated labeling approach may lead to false positives or missing potential refactoring commits. Thus, we perform manual validation on a statistically significant sample size to verify the precision and recall of the automated labeling results. The primary purpose of our manual labeling is to illustrate the overall correctness of the labeled dataset used for training and testing the ML models, rather than to account for every minor change or exception in the detection of the refactoring-related commits. In particular, this automatic labeling approach is designed to build MLRefscanner, not to cover all possible cases comprehensively. Moreover, the automatic labeling approach can have decreased performance when it is applied to new projects or domain-specific projects.

2.3.1 Keyword-Based Labeling. We compile a list of keywords that indicate a refactoring commit (e.g., *modify*, *enhance*, or *clean-up*) used in previous studies [3, 17, 59, 69]. These keywords are typically associated with self-admitted refactoring operations, meaning that developers explicitly mention in the commit messages that refactoring is performed. To create a starting point, we manually investigate each keyword using a small set of 10 sample commits that contain

[1] https://docs.github.com/en/rest/search?apiVersion=2022-11-28
[2] We use PyRef in our data curation, as described in Section 2.3.

the keyword and assess if they consistently represent refactoring commits. This validation process helps us minimize false positive keywords and the manual validation cost. We exclude keywords that do not consistently relate to refactoring operations. For instance, we find that certain keywords, such as *fix* might not always represent refactoring operations. The final set of chosen keywords used to label a refactoring commit includes: *move, refactor, remove, rename, split, clean, improve, unused, cleanup, simplify, restruct, inline, parameterize, consolidate, encapsulate,* and *update*. We then use the keywords to match the commit messages of all the commits of the studied projects: a commit is labeled as a refactoring commit if it contains at least one of the keywords. We use an exact match search, which sets a commit as a refactoring commit if any part of the commit message contains the specified keywords. In the labeling process, we also classify commits that do not alter any executable code (e.g., only changes in the ReadMe file) as non-refactoring commits to ensure that the labeled refactoring commits involve changes to the executable code.

2.3.2 Rule-Based Labeling. Not all refactoring operations are self-admitted; some may be performed without explicit mention in the commit message. As PyRef can achieve a precision of 89.6% and a recall of 76.1% in detecting refactoring operations in ML Python projects, we utilize PyRef [9] to account for refactoring activities not mentioned in the commit message. If PyRef identifies any refactoring operations, we label the commit as a refactoring commit even if there are no specific refactoring keywords in the commit message. The results obtained from PyRef complement the keyword-based labeling results. As PyRef can detect 11 types of refactoring operations including *rename method, add parameter, remove parameter, change/rename parameter, extract method, inline method, move method, pull up method, push down method, change return type, and rename class.* PyRef is used to label our dataset in identifying commits that involve these refactoring operations.

2.3.3 Manual Validation. To assess the accuracy and reliability of our labeling approach, we engage the first author of the article and a third-year undergraduate student in software engineering to conduct manual validation. In cases where there is disagreement, a third-year PhD student in software engineering labels the commit, and consensus is reached among all validators on a final label. Before splitting our dataset into training and testing sets, we randomly select 383 commits from the dataset to ensure that they are not included in either the testing or training sets. This selection is made to maintain a 95% confidence level with a 5% margin of error. Both validators independently assess the commit messages and code changes of the commits and label them based on whether they observe a refactoring operation or not. If they observe one, they label it as a refactoring commit. Cohen's kappa agreement level [14] is found to be 0.64 after the initial labels, indicating a moderate level of agreement [43] between the validators. The validators discuss the initial labels they provided on each commit to reach a consensus on a final label. The final manually validated set includes 76 refactoring commits and 307 non-refactoring commits. By comparing the labels of 383 commits agreed upon by the validators and the labels generated by our automatic labeling approach for the same 383 commits, we achieve a precision of 91% and a recall of 90% using our automatic labeling approach (i.e., using the agreed-upon manual labeling results as the ground-truth). In detail, the automated labeling process identifies five false-positive and five false-negative labels. We observe that the main reason for the false negatives is that the PyRef is incapable of identifying all refactoring types, thus failing to detect refactoring operations in certain commits and reporting them as non-refactoring. Moreover, the reason for the false positives is the misinterpretation of certain keywords, such as *improve*, which do not necessarily describe refactoring commits. For instance, we observe a commit with the message "fix(helper): improve collision in random_port" containing the *improve* keyword, but it is not related to refactoring. Consequently, we use our manually labeled results as the ground truth for further analysis and evaluation of our approach.

Table 1. The Set of Features Utilized in the Study

Feature Type	Name of Features
(1) Textual	TF Vectorized Commit Messages (Vocabulary), Words Count, Sentences Count, Readability
(2) Process	Lines Added, Lines Deleted, Lines Changed, Number of Files, Code Entropy, Refactoring Contribution Ratio, Has Executable File
(3) Code	Avg Count Line, Avg Count Line Blank, Avg Count Line Code, Avg Count Line Comment, Avg Cyclomatic, Count Class Base, Count Class Coupled, Count Class Coupled Modified, Count Class Derived, Count Decl Class, Count Decl Executable-Unit, Count Decl File, Count Decl Function, Count Decl Instance Method, Count Decl Instance Variable, Count Decl Method, Count Decl Method All, Count Line, Count Line Blank, Count Line Code, Count Line Code Decl, Count Line Code Exe, Count Line Comment, Count Stmt, Count Stmt Decl, Count Stmt Exe, Cyclomatic, Max Cyclomatic, Max Inheritance Tree, Max Nesting, Ratio Comment To Code, Sum Cyclomatic

Textual features include vocabulary and features extracted from commit messages; process features encompass details processed from commits; and code features comprise code metrics extracted from the source code before and after each commit code changes.

2.4 Feature Extraction

To identify refactoring-related commits, we consider three dimensions of features that can be extracted from commit logs to effectively detect refactoring commits, including (1) textual features, (2) process features, and (3) code features, as listed in Table 1. Textual features can reveal refactoring commits through commit messages and developer communications, by explicitly mentioning refactoring operations. For example, a commit message may explicitly mention a refactoring-related keyword, such as *refactoring*. Process features are used to measure characteristics of commits that might indicate the presence of refactorings. For example, code entropy may show how diverse the code changes are among different files. Higher entropy potentially indicates the refactoring operations related to code movement across files. Additionally, code features describe the actual code changes and modifications quantified using code metrics, which can reveal underlying patterns of refactoring.

2.4.1 Textual Features. The textual features are listed in Table 1. We first clean up the commit messages to identify the vocabulary in the commit messages. We convert all messages to lowercase to ensure that the capitalization of words does not affect the classification. To address typos and mistakes in commit messages, we utilize the Enchant library [70] to perform spelling corrections and unify word usage, regardless of whether they are mistakenly written or not. We remove URLs, HTML tags, punctuations, emojis, numerical digits from the messages, and stop words (e.g., an, with, or but). Moreover, we stem the commit sentences to their root [10] form to ensure that variations of words (e.g., fixing and fix) are treated as the same. We choose stemming to avoid duplication and improve the generalization of the features. For instance, when using Lemmatization, *refactor, refactoring,* and *refactored* are considered separate features. In contrast, with stemming, we handle all of the variations of the word with a single vector as *refactor*, which prevents our model from being overwhelmed by the explosion of the contextual meanings of unique words in the commits. **Term Frequency (TF)** [67] in commit messages is used to retain the ability to interpret and understand the contributions of individual words in the classification process, which assigns a

value of 0 or 1 based on the presence or absence of a word. We opt for TF instead of TF-IDF (term frequency-inverse document frequency) because TF-IDF tends to assign lower weights to the most repeated words. However, in our case, certain words like *refactor* may appear frequently and should be weighted more. Additionally, we choose not to use vector transformation algorithms like Bert [16] or word2vec [46] as they are not reversible and interpretable. Using such algorithms would prevent us from investigating the root cause and important features that distinguish a commit based on whether it involves refactoring or not. In the vectorization setup, we use 6-grams (i.e., a maximum of six consecutive words as one term) to extract sufficient information from the commit messages, thereby improving our ability to identify potential refactoring activities. The choice of 6-grams is based on a previous study on self-admitted refactorings [3], which identify the maximum number of words in a sentence that can potentially reveal a refactoring commit as 6. In addition, we calculate textual features, such as word count and sentence count for each commit message. To assess the readability of these commit messages, we employ the Flesch reading ease metric [22], which indicates the level of comprehension difficulty in the commit messages.

2.4.2 Process Features. We collect a set of process features that capture various characteristics of commits, as outlined in Table 1. We analyze the modified code blocks and files within each commit using the *git log* information on the cloned repositories to extract features including *lines added, lines deleted, lines changed, has executable file,* and *number of files*. This information helps us better describe the scope of code changes in each commit. The RCR provides information on the experience of each code author performing refactoring-related commits within each project. To calculate the RCR, we collect all labeled commits of each code author before each commit (as described in Section 2.3) and use the formula below:

$$RCR(i) = \frac{Refactoring\ commits\ of\ the\ author\ (i)}{All\ commits\ of\ the\ author\ (i)}. \tag{1}$$

To gain insights into the complexity and structural changes within each commit, we calculate the code changes entropy using Shannon's entropy [64], which has been utilized for measuring the complexity of code changes [30]. For each file within a commit with executable code, we calculate the relative code churn of that file by dividing the code churn of that file by the total code churn of the commit, denoted as $P(x_i)$. Then, we use Shannon's entropy formula to calculate the code change entropy within each commit. The formula for Shannon's entropy is as follows:

$$H(x) = -\sum_{i} P(x_i) log P(x_i). \tag{2}$$

2.4.3 Code Features. To capture information about code changes within each commit, such as code complexity, we calculate code features related to the code changes of each commit. We extract the modified files associated with each commit. Subsequently, we calculate 32 sets of code features in class, function, and code levels, as outlined in Table 1, before and after each commit. We use the Understand[3] tool to calculate code features [44, 73, 75] for the current and previous state of each commit, then we compare the current and previous state of changes and calculate the difference of the code features (i.e., reduction or increase) for each code feature.

2.4.4 Feature Pre-Processing. The existence of highly correlated and redundant features could affect the stability of the model and the interpretation of features [32, 52]. Therefore, on the training set (described in Section 2.5.1), we conduct correlation analysis and redundancy analysis of the process and code features. We don't perform correlation and redundancy analysis on the textual features as it may lead to some loss of information and nuances during model interpretation.

[3] https://scitools.com/

Each textual feature can provide critical insights into why the model has reached a particular decision, and removing correlated textual features can present challenges when interpreting the decision-making process of the model.

— *Correlation Analysis*: Our features do not follow a normal distribution, therefore we utilize Spearman's correlation coefficient to determine the correlation between the computed features. A coefficient > 0.7 indicates a strong correlation [51, 52]. For strongly correlated features, we choose to retain one feature in our model while removing the other. The results of the correlation analysis show that *Number of Files, CountClassBase, CountDeclExecutableUnit, CountLine, CountLineCode, CountLineCodeExe, CountStmt, CountStmtDecl,* and *CountStmtExe* are highly correlated with other features. Therefore, we exclude them from our features.
— *Redundancy Analysis*: R^2 is a metric used to indicate how much of the variance in a dependent variable can be explained by independent variables [47]. To identify redundant features that can be estimated from other features, we use one feature as a dependant and other features as independent features [57] setting a cut-off value of 0.9 [36] for the R^2. Following the removal of highly correlated features, we conduct the redundancy analysis, yet no redundant features are found.
— *Removing Low-Variance Features*: We exclude features with a variance threshold lower than 0.001 in the training set. This threshold implies that the feature value is identical in 99.9% of the data, indicating a minor impact on the data. This step becomes necessary because our initial textual features include 3,433,690 features. By excluding low-variance features, we reduce the feature size to 651, thus focusing on informative features.

Finally, we use min-max normalization [33] on the process and code features to fit them into the same range as TF vectorized textual features, ensuring that the features are rescaled to the range $[0, 1]$. For instance, the number of file changes within each commit could vary from $[0, \infty]$. By utilizing min-max normalization, we rescale it to the range $[0, 1]$.

2.5 Model Construction

To build our ML-based approach, we discuss the model construction steps in the following subsections.

2.5.1 Data Splitting. We randomly split our dataset into a 20% testing set and an 80% training set. To avoid bias, we leave the testing set untouched throughout the entire model-building process, including features pre-processing, undersampling, and hyper-parameter tuning.

2.5.2 Undersampling of Training Data. Our training dataset contains a total of 155,780 refactoring commits and 343,187 non-refactoring commits, resulting in an imbalanced dataset. Class imbalance in our dataset has the potential to skew our classification model towards the majority class (i.e., non-refactoring commits), leading to reduced accuracy [39]. As the size of the dataset is large (i.e., 498,975 records with 651 features), oversampling would significantly increase the computational costs. Moreover, undersampling has been shown to be a better approach compared to oversampling [21, 48]. Hence, we choose to undersample our training dataset. Random undersampling can lead to data loss during the random selection of majority class samples [11]. Therefore, we perform three different NearMiss undersampling approaches [42] and evaluate our models on each, seeking the best-performed undersampling approach. We apply the following undersampling approach to our dataset:

— NearMiss-1: Choose negative (i.e., non-refactoring) samples with an average Euclidean distance [20, 42] closest to the sum of the three closest positive (i.e., refactoring) samples.

—NearMiss-2: Select negative samples that are nearest to the sum of all closest positive examples.
—NearMiss-3: For each positive sample, select the negative examples that are closest.

Therefore, as discussed in Section 3.1, we select 306,197 commits as our final balanced training dataset after undersampling.

2.5.3 Model Training. To train our model, we use 11 different classifiers belonging to three different categories. (1) Linear classifiers: Naive Bayes, **Support Vector Machine (SVM)**; (2) Non-linear classifiers: Decision Tree, k-Nearest Neighbor, Random Forest, **Multi-Layer Perceptron (MLP)**, and CatBoost; and (3) EL: Adaptive Boosting (AdaBoost), Gradient Boosting Machine, XGBoost, and LightGBM [34, 50].

2.5.4 Hyper-Parameter Tuning. To enhance the accuracy of the selected models, we perform hyper-parameter tuning on the training set. As discussed in Section 2.5.2, our training dataset consists of 306,197 records after undersampling and includes the extracted features listed in Table 1. Training the models can be time-consuming, with an average training time of around 20 minutes per model. Therefore, we utilize random search to explore the hyper-parameter space and find the best configuration efficiently. Random search, an efficient and effective hyper-parameter tuning approach, allows us to sample combinations efficiently and often achieves better results in less time compared to other approaches, such as grid search [12].

3 Results

In this section, we provide the motivation, approach, and findings for each of our RQs.

3.1 RQ1. What Is the Performance of Our Approach for Identifying Refactoring Commits in ML Python Projects?

3.1.1 Motivation. Refactoring is applied in practices to increase software maintainability. It is essential to track previous refactoring activities by analyzing commit information so that developers can gain a better understanding of the state of code. This includes monitoring the history and timing of refactoring applied in the history of code, identifying vulnerable code segments that have undergone frequent refactoring operations, and estimating future maintenance needs [8, 13, 55]. The state-of-the-art refactoring detection tools for Python code are relatively limited. For instance, PyRef can detect 11 types of refactoring in Python code. Moreover, existing tools utilize AST pattern-matching to detect refactoring in general repositories, however, the existing tools have not been specifically designed and tested for particular domains, such as ML. With the rapid growth in the number and usage of ML libraries and frameworks, predominantly written in Python, our approach, MLRefScanner is developed to detect commits with more comprehensive refactoring operations within the ML libraries and frameworks written in Python.

3.1.2 Approach. As explained in Section 2, we train and evaluate our refactoring commit detection models using our dataset, which contains 199 ML projects. We assess the overall performance of our model based on precision, recall, AUC, and F1 scores. Precision defines the ratio of true positives to the total positive predictions. Recall measures the ratio of true positives to the actual positive instances. F1 score calculates the harmonic mean of precision and recall. AUC represents the ability of the model to distinguish between classes based on probability [31].

Model Evaluation. We evaluate the performance of our model in the following scenarios:

—*Mixed Projects*: We train our refactoring commit detection models using 80% of the commits of the selected projects, and then we assess the performance of our model on the testing set

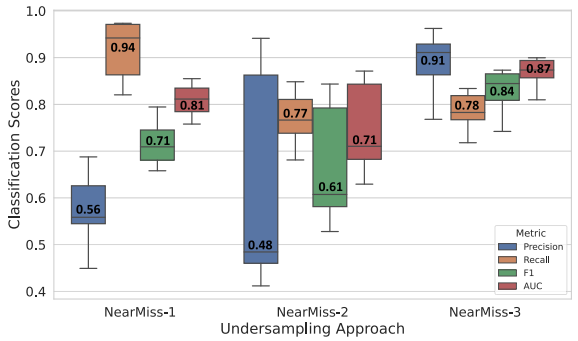


Fig. 2. Compare the results of different undersampling approaches.

which consists of the remaining 20% of all commits in our dataset from different projects altogether.

— *Cross Projects*: Quite often, some projects may not have a sufficient number of historical refactoring commits to train a model. Therefore, we train our model using all but one (n-1 projects) and assess the performance of our models in the remaining project as testing, where the model is tested on the project excluded from the training dataset. We repeat the leaving-one-out approach for all projects.

— *Within Projects*: To maintain consistent results across different projects and prevent large projects from dominating our dataset, we employ a within-projects setting and test our models separately for each project. We train our model using 80% of the commits randomly sampled from each project and evaluate its performance on the remaining 20% of commit messages from the same project. Specifically, we train one model for each project and test it on the same project.

— *Ground Truth*: To ensure the reliability of our model's performance, we test the final model, which is trained on 80% of all commits from all projects, on the ground truth dataset as described in Section 2.3.3.

Manual Analysis of the Identified Refactoring Types in the Mixed-Projects Setting. To gain deeper insights into the capability of MLRefScanner in detecting ML-specific refactoring operations, employing a 95% confidence level with a 5% margin of error, we select a set of 383 commits identified as refactoring by MLRefScanner. Then, we ask one of the co-authors and a fourth-year undergraduate computer science student to conduct a thematic analysis to inspect the refactoring patterns identified by the tool in the chosen refactoring commits. Subsequently, they engage in discussions to compare the patterns they have identified and agree on final refactoring patterns in each commit. During the manual validation, the validators consider a portion of code as a refactoring if it does not alter the external behavior of the code [26]. In manual thematic analysis, Cohen's kappa agreement [14] level in commits involving ML-specific or general refactoring activities is determined to be 0.82 between validators, indicating a perfect level of agreement [43].

3.1.3 *Findings.*

Best Undersampling Approach. NearMiss-3 *undersampling provides the highest AUC (87%) and F1 (0.84%) scores in the mixed-projects setting.* Figure 2 presents the evaluation scores for 11 different classifiers, *NearMiss-3* undersampling achieves the highest median precision (91%) and a median recall of 78%. On the other hand, *NearMiss-1* has a higher median recall (94%), but it experiences a significantly lower precision (56%). Additionally, we manually inspect the

Table 2. The Evaluation Scores of Different Classifiers

Model	Precision	Recall	AUC	F1
LightGBM	0.94	0.82	0.89	0.87
CatBoost	0.92	0.83	0.9	0.87
GBMClassifier	0.90	0.83	0.90	0.87
AdaBoost	0.91	0.82	0.89	0.86
XGBoost	0.96	0.76	0.88	0.85
SVM	0.93	0.77	0.87	0.84
MLPClassifier	0.93	0.77	0.87	0.84
Random Forest	0.84	0.78	0.86	0.81
K-Neighbors	0.88	0.74	0.85	0.81
Decision Tree	0.79	0.81	0.86	0.80
Complement NB	0.77	0.72	0.81	0.74

classification scores in all the classification models and verify the superiority of *NearMiss-3* for all classifiers on our dataset. This can be attributed to the complexity of the decision boundary, as *NearMiss-3* selects non-refactoring samples that are closest to refactoring samples, a technique that has also been demonstrated as effective in prior studies [23, 29, 42]. Consequently, we choose *NearMiss-3* as the undersampling technique.

Best Classification Model. The LightGBM classifier exhibits the best performance in refactoring commit identification, achieving an overall precision of 94%, recall of 82%, and AUC of 89% on the mixed-projects setting. Table 2 presents the details of the scores obtained for different classifiers using *NearMiss-3* undersampling. Our findings demonstrate that both LightGBM and CatBoost classifiers outperform other approaches, with CatBoost achieving a precision of 92% and a recall of 83% in the mixed-projects setting. Given our objective to maximize precision (i.e., minimize false positives), we select LightGBM as the nominated model and the best-performing classifier for the remainder of our study.

Our approach can not only accurately detect refactoring commits when trained and tested in the same projects (i.e., within-projects) but also in projects that have not been seen before, even if they may exhibit different characteristics (i.e., cross-projects). In within-projects validation, we achieve a median precision of 94%, a median recall of 82%, and a median AUC of 87%. In the cross-projects validation, we achieve a median precision of 86%, a median recall of 80%, and a median AUC of 90%. The results of the within-projects and cross-projects validations are shown in Figure 3. Furthermore, we achieve a precision of 90%, a recall of 81%, and an AUC of 90% on the ground truth dataset.

MLRefScanner can detect commits with ML-specific or general refactoring operations that cannot be detected by state-of-the-art refactoring detection tools. MLRefScanner can detect commits involving both general refactoring operations at the class, method, and variable levels, as well as ML-specific refactoring operations related to data handling, optimization, and ML pipelines. Through manual thematic analysis, we identify a set of ML-specific and general refactoring patterns in the commits detected by MLRefScanner that cannot be identified using state-of-the-art AST pattern-matching refactoring detection tools. ML-specific refactoring patterns are typically related to dataset handling, data reading/writing operations, mathematical computations, and ML model parameter adjustments. On the other hand, general refactorings are broader refactoring operations that do not adhere to a regular pattern; instead, they mainly rely on domain knowledge. For instance, code optimization involves replacing a block of code with a simpler and more understandable alternative. The explanation and the frequency of application for the extended refactoring types are provided in Table 3.

MLRefScanner is trained on Python projects but is not limited to Python source files and code. Through manual thematic analysis, we observe that 22 out of 384 commits with source files other

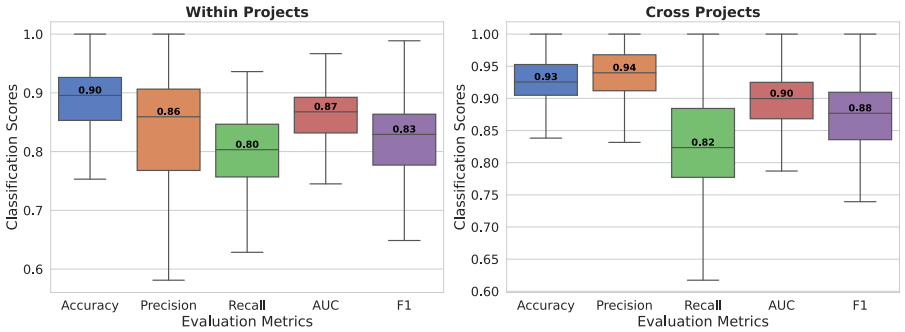


Fig. 3. Compare the performance of our approach using cross- and within-project validations.

Table 3. The List of Refactoring Types That Can Be Identified by MLRefScanner but Cannot Be Identified by State-of-the-Art Pattern-Matching Detection Tools

	Type	Description	Frequency
ML-specific	Data Handling Optimization	Optimizing data reading/writing from/to the dataset.	14.4%
	Model Initialization Refinement	Modify the hyper-parameter initializations in the models.	04.8%
	Resource Allocation Optimization	Adjusting hardware usage parameters.	04.8%
	Data Path Management	Managing dataset/plots/models storage paths.	03.8%
	Data Type Clarification	Converting data types from one to another (e.g., from NumPy to DataFrame).	02.9%
	Data Presentation Enhancement	Refactoring data visualization code.	02.9%
	Mathematical Operation Refactoring	Changing mathematical calculation to a simpler form.	00.7%
General	Code Cleanup	Removing unused code or files.	16.7%
	Code Simplifications	Replacing code with more simplified alternatives.	13.5%
	Import/Export Optimization	Removing or relocating unused dependencies.	10.9%
	Logging Enhancement	Improving log messages for user understanding.	10.5%
	Requirement/Configuration Update	Updating program requirements or configurations.	08.4%
	Condition Simplification	Simplifying complex conditional statements.	03.0%
	File/Dependency Path Refinement	Optimizing/updating dependency/file path.	02.5%

than Python have been detected as refactoring commits. This is because MLRefScanner is trained not only using code features but also a set of process and textual features that are independent of programming languages. Our refactoring detection model uses 651 different features (i.e., keywords, code features, and process features) to decide on a commit. It learns from the automatically labeled dataset to identify refactorings in different projects that don't necessarily follow the rules or keywords used in the labeling process. For example, a commit with the message "add base class for TensorBoard handlers and refine code formatting" is labeled as a non-refactoring commit based on our automatic labeling approach. However, our classifier identifies this as a refactoring commit. This is because our classifier considers not only the entire vocabulary in the commit message (e.g., *add* and *format*) but also incorporates code features and other process features into its decision-making process. Therefore, we assume that MLRefScanner would be more generalizable to unlabeled data than directly using the labeling approach.

To verify this assumption and assess the generalizability of our approach, we manually evaluate 8% of the testing set where the labeled results by the automatic labeling approach differ from our classification approach. The first author of the article and another graduate student in software engineering independently validate 30% of 382 randomly selected commits, following the same approach as Section 2.3.3, for refactoring, achieving 83% precision and 81% recall, with a Cohen's kappa of 0.90. Due to the excellent agreement and the limited resource available for manual evaluation of the rest of the sampled dataset, the first author of the article continues to label the

remaining commits. For the 382 sampled commits, MLRefScanner achieves 82% precision and 80% recall, while the automatic labeling approach has only 18% precision and 37% recall, confirming the validity of our assumption.

MLRefScanner can accurately detect refactoring commits within the ML technical domain (i.e., ML projects) in mixed-projects, within-projects, cross-projects, and ground-truth settings. MLRefScanner achieves 94% precision, 82% recall, and 89% AUC on the testing set of the selected ML projects in the mixed-projects setting. MLRefScanner is capable of detecting ML-specific (e.g., data processing optimization) and a set of new refactoring types (e.g., Code Cleanup) that are dismissed by state-of-the-art tools relying on AST pattern-matching algorithms.

3.2 RQ2. What Are the Main Features for Explaining the Refactoring Commits?

3.2.1 Motivation. In the first RQ, we propose a classifier capable of identifying refactoring commits in the ML technical domain. It is important to explain the outcomes from the classifier decisions on detecting refactoring commits. By understanding key features, we can provide practitioners, especially tool makers, and researchers, with insights into the characteristics of refactoring commits.

3.2.2 Approach.

Importance of Each Feature Type. We assess the impact of each feature type (i.e., textual, process, and code) listed in Table 1 on the decision-making process of our best-performing model (i.e., LightGBM). To achieve this, we train separate LightGBM models using textual, process, and code data individually on the ML training set and measure the performance of each model on the ML testing set separately. This enables us to (1) indicate the importance of each feature type and (2) demonstrate how each feature type contributes to the identification of refactoring commits.

Individual Feature Importance. We select the best-performing trained classifier (i.e., LightGBM) to compute the importance of each feature by assessing its frequency and significance in splitting the decision trees during the training process. However, the number of splits in the decision tree doesn't interpret how each feature contributes more towards predicting specific prediction (i.e., refactoring or non-refactoring commit) and the model remains a black box. Therefore, we leverage the LIME technique [60].

LIME enables us to explain the features of individual instances in our dataset and assigns weights to each feature, indicating their impact on the decision-making process of the model. Using LIME, we gain insights into the model's behavior at the instance level and can better understand how each feature affects the predictions of our model. This approach enhances the transparency and interpretability of the decision-making process of our model.

Explaining Feature Contributions with LIME. We first use the classifier's splitting score, and then we apply LIME to extract the weight of each feature for each instance in our testing set. A negative weight reported for a feature indicates that the presence or increase in that feature contributes to the decision-making process toward the negative class (i.e., refactoring), while a positive weight indicates that the increase or presence of that feature is associated with the decision-making process favoring the toward class (i.e., non-refactoring). Lastly, a neutral weight implies that the feature does not independently influence the decision-making process but rather relies on other features for classification. We train LIME on our training set and then apply it to every instance in the testing set. We use the median weights of features, and we assign a positive, negative, or neutral weight to each feature, indicating the extent to which a feature is generally associated with refactoring commits.

3.2.3 Findings. Overall, textual features play the most important role in identifying refactoring commits. However, the addition of process and code features to textual features results in an improvement

Table 4. The Results of the Trained Classifier on Each Metric Type and Overall

Feature Type	Precision	Recall	AUC	F1
Textual	0.87	0.71	0.83	0.78
Process	0.52	0.73	0.71	0.60
Code	0.53	0.41	0.62	0.46
Overall	0.94	0.82	0.89	0.87

of 7% in precision, 11% in recall, and 7% in AUC. The results of inspecting the impact of different types of features (i.e., process, code, and textual) on our model are shown in Table 4. The significant improvement in recall brought by the process and code features enables the identification of refactoring commits that may not be explicitly mentioned in the commit messages.

The appearance of certain vocabulary keywords in a commit message indicates the existence of refactoring in that commit. The most influential keywords for classifying a commit as refactoring include *update, remove, refactor, improve, move, rename, clean, cleanup, add, simplify, split, unused, restructure, option, name, parameter,* and *argument*. Specifically, some textual features may indicate a general refactoring operation in a commit (e.g., *refactor*), while others may be more specific, such as *rename*, which indicates the renaming of a variable, method, or class at some level of the code.

Past refactoring contributions of the developers, code complexity features, and code changes are the most common indicators of refactoring. More specifically, increased values of the process and code features such as *refactoring contribution, lines deleted, lines added, code entropy, CountLineCodeDecl, SumCyclomatic, AvgCountLineCode, MaxCyclomatic, CountLineComment,* and *CountLineBlank* play a significant role in influencing the decision of our model towards classifying a commit as refactoring.

The presence of certain vocabulary keywords in a commit message indicates the absence of refactoring in that commit. Keywords such as *readme, doc,* and *fix* are identified as characteristics that lead the model to make a non-refactoring decision. Furthermore, other features, including *Executable File*, *CountDeclFunction*, *Lines Changed*, *Readability*, *words count*, *RatioCommentToCode*, and *ad* vocabulary keyword, are considered dependent features that assist other features in the decision-making process. They do not directly influence the model's decision but contribute to the overall classification outcome in conjunction with other features. They enhance the model's predictions by adding additional context that complements other features, thereby playing a supportive role. The results of our feature importance approach with at least 10 splits in the decision tree are listed in Table 5 (the full list of features is included in our replication package).

Our approach provides a complementary set of important textual, process, and code features, helping practitioners, researchers, and tool makers understand the significance of different features in identifying refactoring commits.

Textual features, which capture the characteristics of commit messages, play a crucial role in detecting refactoring commits. Additionally, past refactoring contributions of developers, code changes, and code complexity features significantly contribute to identifying refactoring commits that lack indicators in their commit messages.

3.3 RQ3. Can We Leverage Commit Information to Complement Existing Keyword-Based and Rule-Based Approaches?

3.3.1 Motivation. Historical information of software has been used to identify refactorings in prior studies [3, 9, 17, 38, 59]. Some approaches use a specific set of keywords to determine whether

Table 5. Most Important Features in Our Classifier, Each with at Least 10 Splits in the Decision Tree

Feature Type	Feature Name	Importance	Refactoring	Example / Description
Textual	CMV:updat (update)	170	✔	The return type of the method call is updated (Change Return Type)
	CMV: remov (remove)	145	✔	The parameters of the method is/are removed (Remove Parameter)
	CMV:refactor	78	✔	Refactored the contributor guide into many more sub-sections and made a start on some of them
	CMV:improv	50	✔	Small name update to improve clarity in func_wrapper (Change Function Name)
	CMV:move	45	✔	Move dtype functions from general to dtype jax (move method)
	CMV:renam (rename)	43	✔	The method is_array is renamed to is_native_array (Rename Method)
	Readability	41	—	The Flesch commit message readability metric [22] (described in Section 2.4.2)
	Words Count	41	—	Number of words in the commit message
	CMV:clean	38	✔	Clean up databricks scripts (Multiple Refactoring Operations)
	CMV:cleanup	32	✔	Fixed typos in code cleanup process (Change Variable Name)
	CMV:add	29	✔	The parameters are added to the method get_relevant_items_by_threshold (Pull Up Parameters)
	CMV:simplifi (simplify)	23	✔	Simplification of predict function through parameter (Remove Parameter)
	CMV:split	22	✔	Split configuration and modeling files (Move Method)
	CMV:unus (unused)	19	✔	Remove unused args parameter from PreTrainedConfig.from_pretrained (Remove Parameter)
	CMV:readme	15	✘	Rename the readme file
	CMV:restructur (restructure)	14	✔	Restructured function (Move Method)
	CMV:option	14	✔	Add options for save_as (Add Parameter)
	CMV:doc	12	✘	Update document of standalone deployment
	CMV:name	12	✔	Rename some function (Rename Method)
	CMV:ad	11	—	Import optional modules *ad hoc* in the runner
	CMV:paramet (parameter)	10	✔	Change parameter names (Rename Parameter)
	CMV:fix	10	✘	Fix parallel execute for bug
	CMV:argument	10	✔	Removed unused argument (Remove parameter)
Precess	Refactoring Contribution	239	✔	The developer refactoring contribution (described in Section 2.4.2)
	Lines Deleted	140	✔	The number of lines deleted in a commit
	Has Executable File	108	—	If a commit contains executable code
	Lines Added	94	✔	The number of lines added in the commit
	Code Entropy	88	✔	The code entropy of each commit (described in Section 2.4.2)
	Lines Changed	49	—	The number of lines changed in a commit (Lines Added - Lines Deleted)
Code	CountLineCodeDecl	87	✔	Count of lines that consist of declarative source code [63]
	CountDeclFunction	51	—	Methods declared within a class accessible exclusively via an instance of that class [63]
	SumCyclomatic	42	✔	The total weight of methods belonging to a class [63]
	AvgCountLineCode	36	✔	Mean number of lines containing source code within all nested functions or methods [63]
	CountLineComment	30	✔	Mean number of lines containing comments for all nested functions or methods [63]
	CountLineBlank	24	✔	The number of black lines of code [63]
	RatioCommentToCode	22	—	Proportion of lines containing comments compared to lines containing code [63]
	MaxCyclomatic	19	✔	The highest cyclomatic complexity among all nested functions or methods [63]

Features marked with ✔ indicate refactoring features, ✘ indicate non-refactoring features, — signifies neutral features, and features with the CMV prefix indicate vocabulary features. For the vocabulary features, we provide an example of a commit message containing that feature and set the possible refactoring operation in parentheses. For other features, we include a description of the feature.

a commit is related to refactoring or not [3, 17, 38, 59]. Moreover, certain tools, like PyRef [9], utilize pattern-matching on the AST level to ascertain the presence of refactoring in a commit. In this RQ, we first compare our approach with state-of-the-art tools and methods that identify refactoring commits to evaluate the effectiveness of our approach. Moreover, we aim to determine whether ensembling our approach with PyRef, the state-of-the-art and rule-based approach, aids in improving refactoring commit detection. This analysis helps us provide a more effective solution by complementing the state-of-the-art tools to identify refactoring commits in ML projects mainly written in Python.

3.3.2 *Approach.*

Comparing MLRefScanner with Baselines. We compare the output of our approach with the following keyword-based and rule-based approaches by measuring the performance metrics (i.e., precision, recall, F1, and AUC) on the testing set in the mixed-projects setting. For the keyword-based approaches [3, 17, 38, 59], state-of-the-art studies have used or identified certain keywords to identify refactoring commits in the commit history. We use the same keywords as the state-of-the-art approaches and search through the entire commit history to identify refactoring commits based on the keywords provided by the state-of-the-art approaches.

For the rule-based approach [9], we run PyRef accordingly to get refactoring commits. As discussed in Section 3.3.3, PyRef achieves the highest precision (100%) compared to its opponents,

Table 6. Comparison of Results Obtained from MLRefScanner and State-of-the-Art Approaches

Approach	Category	Precision	Recall	AUC	F1
MLRefScanner	ML Based	94%	82%	89%	87%
PyRef	Rule Based	100%	33%	67%	50%
AlOmar et al.	Keyword Based	39%	39%	56%	39%
Ratzinger et al.	Keyword Based	84%	26%	62%	40%
Zhang et al.	Keyword Based	48%	29%	57%	36%
Kim et al.	Keyword Based	93%	25%	62%	39%

but it suffers from low recall. This motivates us to consider ensembling our approach with PyRef to potentially improve the low recall of the PyRef.

Ensembling MLRefScanner with PyRef. In our approach, we obtain a best-performing classifier that utilizes code, textual, and process features to identify whether a commit is a refactoring commit or not. Conversely, PyRef employs pre-defined rules to identify different types of refactoring and, consequently, refactoring commits. To potentially improve the recall in predicting refactoring commits, we propose an EL [61] method to combine the results from the best-performing classifier (i.e., MLRefScanner) with PyRef and measure if we can complement PyRef and improve its low recall. Through this ensemble analysis, both our classifier and PyRef vote on whether a commit is refactoring or not. We test and apply unanimous (i.e., both PyRef and our approach detect refactoring) and majority (i.e., at least one of PyRef or our approach detects refactoring) voting schemes and measure the enhancements in refactoring commit detection. This provides a more robust and comprehensive decision-making process.

3.3.3 Findings. MLRefScanner outperforms the state-of-the-art rule-based and keyword-based approaches, achieving, on average, 36% higher precision, 52% higher recall, 28% higher AUC, and 30% higher F1 scores compared to previous keyword-based approaches [3, 17, 38, 59] *and 6% lower precision, 49% higher recall, and 22% higher AUC, compared to the rule-based state-of-the-art approach* [9] *on the testing set.* Table 6 shows the results of our approach compared to other refactoring detection approaches on our ML testing set in the mixed-projects setting. MLRefScanner achieves a recall of 82%, surpassing PyRef [9] at 33%, AlOmar et al. [3] at 39%, Ratzinger et al. [59] at 26%, Zhang et al. [17] at 29%, and Kim et al. [38] at 25%. This indicates that previous approaches overlook a significant number of refactoring commits, making MLRefScanner superior to state-of-the-art methods in detecting all possible refactoring commits.

Our approach can complement the low recall of PyRef with a 60% increase in its recall in the majority vote setting. Despite a 6% lower precision, our approach achieves a 49% higher recall compared to PyRef, indicating our approach can capture a broader window of refactoring commits in the development history. Table 7 shows our ensembling results by combining our approach and PyRef. Our ensemble approach achieves a precision of 95%, recall of 99%, an AUC of 98%, and an F1 score of 97% in the majority vote setting. Therefore, we can effectively complement and assist PyRef in finding the refactoring commits that might have otherwise been overlooked.

On average, our approach, MLRefScanner, provides 21% higher precision, 52% higher recall, 28% higher AUC, and 45% higher F1 score compared to state-of-the-art rule-based and keyword-based approaches. MLRefScanner can complement PyRef's low recall and significantly increase it by 60% in refactoring commit detection, motivating refactoring tool makers to consider more features in the refactoring detection process.

Table 7. Comparison of Results Obtained from Our Classifier, PyRef, and Ensemble Approaches

Method	Precision	Recall	AUC	F1
PyRef [9]	100%	39%	70%	56%
MLRefScanner	94%	82%	89%	87%
Unanimous Voting	100%	21%	61%	35%
Majority Voting	95%	99%	98%	97%

4 Discussion on the Generalizability of Our Approach

As discussed in Section 3.1, MLRefScanner performs well within ML projects and achieves high precision and recall under different testing settings in the ML technical domain compared to rule-based and keyword-based state-of-the-art approaches. In this section, we want to investigate the potential applicability of MLRefScanner across diverse technical domains by assessing its performance on general (i.e., non-ML) Python projects. For this purpose, we assemble a testing dataset comprising a variety of general Python projects, aiming to evaluate the adaptability of MLRefScanner beyond ML applications. We follow similar project selection criteria used for choosing ML projects to select the general Python projects (described in Section 2.2). We do not restrict the number of commits for general projects, allowing us to evaluate the effectiveness of our approach on both smaller projects with a limited number of commits and larger projects. This process results in a set of 12,665 general projects. Subsequently, we randomly select the same number of projects as in the ML category (i.e., 199 projects) to evaluate the performance of our approach in general Python projects. We manually validate the general Python projects to ensure that projects containing only documentation are not included. Following this, we validate and verify our model to further assess the generalizability in the general Python projects. We assess the performance of our model in the context of general Python projects under the following scenarios:

- *Combined General Projects*: Using a testing dataset comprising 199 general Python projects, we assess the performance of our model trained on ML projects across the entire dataset of general projects. This testing dataset includes all commits from all 199 general projects.
- *Individual General Projects*: Using individual general Python projects as testing datasets, we individually test our model trained using ML projects, on all commits of each general Python project. This approach allows us to assess the effectiveness of our model within the specific context of each project.

MLRefScanner can be extended to projects within other technical domains. In the combined general projects setting, our model successfully achieves a precision of 99%, a recall of 93%, and an AUC of 96% on general Python projects. In the individual general projects setting, we attain a median precision of 99%, a median recall of 100%, and a median AUC of 100%. Maintaining high performance across technical domains is attributed to our use of features extractable from any Python repository, which are not specific to any particular domain. The performance of our approach can generalize from one technical domain (i.e., ML projects) to other domains (i.e., general Python projects). Practitioners and researchers can apply our findings in various contexts, whether they are working on traditional Python projects or those involving ML while maintaining a high level of performance.

5 Implications

In this section, we provide the implications of our study for managers, developers, and tool-makers.

Enhancing Refactoring Evolution Monitoring in the ML Technical Domain. Understanding the refactoring process can help improve software design [62]; consequently, accurately classifying refactoring commits can enhance the monitoring of refactoring activities during software evolution

and provide better tracking mechanisms for stakeholders [40]. *ML Software managers and developers* can effectively track the refactoring effort and gain insights into the volume of refactoring activities. This information allows them to review refactoring efforts and make well-informed decisions. For instance, they can identify areas of the code that are consistently being refactored, indicating potential issues that require prompt attention and action. Furthermore, developers' refactoring efforts can be observed and assessed, increasing motivation for refactoring. Existing tools [9] and approaches [3, 17, 38, 59] for identifying refactoring activities suffer from low recall, either due to their limited ability to detect various types of refactoring or their primary reliance on text search-oriented methods. However, our approach facilitates effective tracking of refactoring commits in the history of ML Python projects with a precision of 94% and a recall of 82%. Therefore, we encourage software teams to utilize our approach to track possible refactoring activities within their ML projects.

Assisting ML Researchers and Refactoring Tool Makers in Filtering out a Substantial Portion of Commits That Are Unrelated to Refactoring. Distinguishing between refactoring and non-refactoring commits can be challenging for developers [27]. By drawing a clear distinction between refactoring and non-refactoring commits, our approach enables ML *researchers and developers* interested in refactoring to focus their efforts on commits that involve refactoring. We recommend researchers and developers refer to this targeted approach for designing automated tools and conducting in-depth studies on refactoring activities. For instance, instead of solely relying on PyRef [9], they can incorporate PyRef with our approach to target refactoring commits more effectively.

Providing a Set of Features Significantly Correlated with Refactoring Activities. By offering a set of features that exhibit a significant correlation with refactoring activities, we highlight the key attributes that encompass textual, process, and code feature aspects and warrant priority in the study of refactoring activities. *Toolmakers and researchers* can incorporate these features while studying refactoring evolution and related research. By providing an extensive list of top features related to refactoring and non-refactoring commits, we aim to help practitioners understand how developers document their refactoring activities (textual features), the effect of external process features (e.g., developer refactoring ratio), and code features such as cyclomatic complexity, which can be closely related to refactoring activities.

Demonstrating a Complementary Role of Other Features in Refactoring Identification. The well-known refactoring detection tools PyRef [9] in Python and Rminer [71] in Java utilize AST pattern-matching algorithms. In this work, we showcase the effectiveness of incorporating source code features alongside pattern-matching to enhance the accuracy and broaden the scope of refactoring types that can be identified. We demonstrate how ensembling PyRef, a rule-based approach, with our ML-based approach can significantly increase the recall of PyRef by 60% in refactoring commit detection. Therefore, we recommend *tool-makers* consider employing ensembling different techniques when designing refactoring identification tools for analyzing project development history.

Adoption of MLRefScanner. MLRefScanner is an ML-based approach that extracts features from commit logs and constructs ML models to detect refactoring commits. We shared our ML-based pipeline, including our dataset and ML model construction, and provided guidelines for data extraction and pre-processing for refactoring detection in projects. This helps practitioners deploy the approach with minimal effort. In addition, MLScanRef performs the best using the LightGBM algorithm which is a highly efficient model. For optimal performance when building MLRefSscanner, we recommend having at least 5 GB of memory available and a modern multi-core processor (e.g., Intel i5/i7 or AMD Ryzen).

6 Threats to Validity

In this section, we discuss the threats to the validity of our study.

Threats to construct validity concern the data preparation, feature selection, and model construction methods employed in our study. In dataset preparation, we utilize a set of textual, process, and code features to characterize the commit messages. Nevertheless, incorporating additional features has the potential to further enhance the accuracy of our classification model. In the model construction phase, we utilize random search for hyper-parameter tuning in the model construction due to its cost-effectiveness, other hyper-parameter tuning approaches could potentially further improve the accuracy of our classifier. To assess the performance of the models across commits with varying numbers of refactoring operations, we conduct a sensitivity analysis by adjusting the labeling threshold of the number of refactoring operations in a commit obtained from PyRef. We measure the performance of the models with different thresholds to evaluate the performance of the models in identifying refactoring commits. As the threshold for refactoring operations increases, the recall of the model drops, meaning that the classifier dismisses the commits with fewer numbers of refactoring operations. This observation further supports the efficacy of the selected threshold (i.e., the existence of at least one refactoring operation in a commit) in identifying refactoring commits. Although we considered a large number of different projects to ensure diversity in the commit message keywords, our automatic labeling approach may sometimes still generate false positives or false negatives, especially when there are ambiguous keywords or synonyms. To understand the validity of our automated labeling, we manually verified the automated labeling results by examining a statistically representative sample from the entire commit dataset, demonstrating the high precision and recall of our automated labeling approach. This approach was selected because validating all commits was not feasible with our available human resources. However, we acknowledge that this labeling may be potentially imprecise and may not be generalized to specific domains or projects using different commit message conventions.

Threats to internal validity indicate the validity of methods utilized in our study. In the experiment setup, we opt to use TF to vectorize commit messages [41] instead of using other vectorizing approaches, such as Bert [16] and word2vec [46], due to their lack of transparency and irreversibility. While we acknowledge that using such vector transformation algorithms could potentially enhance our commit message analysis, we prioritize the interpretability and reversibility of the chosen approach in order to provide explainability of our approach. In the process of labeling validation and ground-truth preparation, we obtain a Cohen's kappa agreement level of 0.64, indicating a moderate level of agreement between the validators. However, the two validators have discussed the results of the manual validations meticulously to reach a conclusion on each commit and achieve a consensus on the final label, ensuring the avoidance of bias in the labeling process. In comparing our approach with state-of-the-art keyword-based approaches [3, 17, 38, 59], we utilized the set of keywords provided by these approaches to detect refactoring-related commits. However, we acknowledge that the recommended keywords from these state-of-the-art approaches may be specifically relevant to their studied datasets.

Threats to external validity indicate the generalizability of our approach. The textual features that our classification model is trained on are tailored to English languages. The reported performance scores reflect the performance of our classification model on repositories that predominantly use English to document their commits. As a result, when applied to non-English repositories, the performance of our approach may decrease. We validate our approach on 199 general English Python projects to ensure its generalizability across various Python projects. Since our labeling and ML approaches rely on the quality of the commit messages written by developers, the performance of our ML-based approach and our evaluation results may be challenged if developers do not use

informative commit messages, potentially leading to incorrect labels and prediction results in some cases.

7 Related Works

In this section, we review the literature on refactoring detection approaches in the commit messages.

Refactoring Detection Tools. A variety of tools have been proposed to detect refactoring operations in the history of software [9, 49, 65, 66, 72, 72]. These tools primarily rely on algorithms that match patterns and rules to identify refactoring operations between two versions of the source code. Silva et al. [66] introduce RefDiff 2.0, a multi-language refactoring detection tool, which supports 10 types of refactoring operations in Java and JavaScript and nine types of refactoring operations in C languages, achieving an overall precision and recall ranging from 88% to 91% across different languages. Moghadam et al. [49] propose RefDetect, which uses string alignment algorithms to detect 27 types of refactoring in Java and C++. RefDetect achieves an overall precision and recall of 91% and 85% in Java and 96% and 94% in the C++ language. Tsantalis et al. [71, 72] introduce Rminer, which overcomes coverage constraints by applying pattern-matching within the AST to detect refactoring operations. The current version of Rminer (2.4) can identify up to 99 refactoring operations in the history of Java programs with a precision of 100% and a recall of 94%. Using a method similar to Rminer, Shiblu [65] introduce jsDiffer, which is capable of identifying 18 types of refactoring for JavaScript with a precision of 96% and a recall of 44%. Sagar et al. [62] use text and code features to detect six types of refactoring operations in the history of Java code and achieve an average precision 81% and an average recall 75% in refactoring detection. Atwi et al. [9] use a similar approach to Rminer and introduces PyRef. Despite their performance measures, the primary weakness of tools proposed for languages other than Java is a lack of coverage, meaning they often fail to discover all types of refactorings in the development history. In this work, we aim to propose a prototype tool to complement and enhance refactoring detection in the Python language, which is popular in both the ML and non-ML development communities.

Self-Admitted Refactoring Patterns. AlOmar et al. [3] conduct an experimental study and identify a set of SAR patterns potentially used to describe refactoring commits. They find that commits with SAR patterns make a more significant contribution toward major refactoring activities. Ratzinger et al. [59] identify refactoring commits by investigating keywords like "refactor" and its extensions, such as "needs refactor" in the commit messages. Similar to Ratzinger et al. [59], Stroggylos and Spinellis [69] examine commit messages, searching for words stemming from "refactor" and label them as refactoring commits. Di et al. [17] distinguish the refactoring commits using 22 keywords adopted from Fowler [25] refactoring categories. AlOmar et al. [5] investigate self-affirmed refactoring commits by leveraging the keywords obtained from their previous work [3] to investigate self-affirmed refactoring commits in Java projects. Kim et al. [38] conduct a survey study, asking developers what keywords they use to indicate refactoring in their commits. They then utilize 10 keywords to detect refactoring commits from version history and correlate them with code features. Different from the aforementioned approaches, we employ ML techniques to detect refactoring activities within the commit history. We compare our proposed approach with the previous keyword-based methods for detecting refactoring or self-admitted refactoring commits. We demonstrate how keyword selection and pattern-matching in commit messages cannot detect all possible refactoring commits.

Refactoring Detection in Python Code. Refactoring detection tools for Python are limited, and to the best of our knowledge, code conversion [19] and AST pattern-matching [9] are the only approaches available. Due to the differences in syntax between Python and Java, certain code information may be lost, such as the lack of variable type definition in Python. Therefore, code conversion may not be a practical solution. Dilhara et al. [19] convert Python code into Java and

then use Rminer [71] to detect the refactoring operations in Python projects. Atwi et al. [9] use the same pattern-matching in AST as Rminer [71] does and introduce PyRef, which can identify up to 11 types of refactorings, achieving an overall precision of 89.6% and a recall of 76.1%. They examine the approach by Dilhara et al. for converting Python code to Java and observe a precision of 71.74% and a recall of 50.13% for their approach. PyRef is the superior refactoring detection tool in Python showing higher precision and recall compared to its opponent. In this study, we compare our refactoring detection approach with PyRef and demonstrate a 49% higher recall in our model compared to PyRef.

8 Conclusion

In this study, we introduce our approach, MLRefScanner, which is capable of identifying refactoring commits in the history of ML projects. We conduct extensive experiments on 199 ML Python projects and verify the performance of MLRefScanner in various testing scenarios. MLRefScanner achieves a precision of 94%, a recall of 82%, and an AUC of 89%, and a minimum set of features is provided for distinguishing refactoring commits. Our approach outperforms state-of-the-art refactoring detection methods, with an average of 21% higher precision, 52% higher recall, 28% higher AUC, and 45% higher F1 scores in the mixed-projects scenario. Moreover, ensembling MLRefScanner with PyRef, the most recent and accurate refactoring detection tool, showcases a 60% improvement in its overall recall. MLRefScanner successfully identifies refactoring activities that were previously overlooked by state-of-the-art methods and can detect commits involving ML-specific refactoring operations. MLRefScanner enhances the refactoring detection in ML technical domain by addressing the issue of low recall present in existing state-of-the-art approaches. Moreover, we contribute to the refactoring research community with an extensive dataset that includes refactoring activities from 199 ML repositories for further refactoring analysis. In the future, we plan to extend and fine-tune our approach to enable the detection of specific types of refactorings, such as pull-up method and move-class, by mining software development histories.

Acknowledgments

We want to express our appreciation to Jonathan Cordeiro and Yiping Jia at Queen's University for helping to manually validate our results. Their commitment was really helpful in confirming the reliability and precision of our findings.

References

[1] Sabah A. Abdulkareem and Ali J. Abboud. 2021. Evaluating Python, C++, Javascript and Java programming languages based on software complexity calculator (halstead metrics). *IOP Conference Series: Materials Science and Engineering*, 1076, 012046.

[2] Jehad Al Dallal and Anas Abdin. 2017. Empirical evaluation of the impact of object-oriented code refactoring on quality attributes: A systematic literature review. *IEEE Transactions on Software Engineering* 44, 1 (2017), 44–69.

[3] Eman AlOmar, Mohamed Wiem Mkaouer, and Ali Ouni. 2019. Can refactoring be self-affirmed? An exploratory study on how developers document their refactoring activities in commit messages. In *Proceedings of the 2019 IEEE/ACM 3rd International Workshop on Refactoring (IWoR '19)*. IEEE, 51–58.

[4] Eman Abdullah AlOmar, Hussein AlRubaye, Mohamed Wiem Mkaouer, Ali Ouni, and Marouane Kessentini. 2021. Refactoring practices in the context of modern code review: An industrial case study at Xerox. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '21)*. IEEE, 348–357.

[5] Eman Abdullah AlOmar, Mohamed Wiem Mkaouer, and Ali Ouni. 2021. Toward the automatic classification of self-affirmed refactoring. *Journal of Systems and Software* 171 (2021), 110821.

[6] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software engineering for machine learning: A case study. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '19)*. IEEE, 291–300.

[7] Arooj Arif and Zeeshan Ali Rana. 2020. Refactoring of code to remove technical debt and reduce maintenance effort. In *Proceedings of the 2020 14th International Conference on Open Source Systems and Technologies (ICOSST '20)*. IEEE, 1–7.

[8] Guisella A. Armijo and Valter V. de Camargo. 2022. Refactoring recommendations with machine learning. In *Anais Estendidos do XXI Simpósio Brasileiro de Qualidade de Software*. SBC, 15–22.

[9] Hassan Atwi, Bin Lin, Nikolaos Tsantalis, Yutaro Kashiwa, Yasutaka Kamei, Naoyasu Ubayashi, Gabriele Bavota, and Michele Lanza. 2021. PyRef: Refactoring detection in Python projects. In *Proceedings of the 2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM '21)*. IEEE, 136–141.

[10] Vimala Balakrishnan and Lloyd-Yemoh Ethel. 2014. Stemming and lemmatization: A comparison of retrieval performances. *Lecture Notes on Software Engineering* 2 (Jan. 2014), 262–267. DOI: https://doi.org/10.7763/LNSE.2014.V2.134

[11] Gustavo E. A. P. A. Batista, Ronaldo C. Prati, and Maria Carolina Monard. 2004. A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD Explorations Newsletter* 6, 1 (2004), 20–29.

[12] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *Journal of Machine Learning Research* 13, 2 (2012), 281–283.

[13] Diego Cedrim, Alessandro Garcia, Melina Mongiovi, Rohit Gheyi, Leonardo Sousa, Rafael De Mello, Baldoino Fonseca, Márcio Ribeiro, and Alexander Chávez. 2017. Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects. In *Proceedings of the 2017 11th Joint Meeting on foundations of Software Engineering*. ACM, New York, NY, 465–475.

[14] Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences* (2nd. ed.). Routledge, New York, NY. DOI: https://doi.org/10.4324/9780203771587

[15] Elizamary de Souza Nascimento, Iftekhar Ahmed, Edson Oliveira, Márcio Piedade Palheta, Igor Steinmacher, and Tayana Conte. 2019. Understanding development process of machine learning systems: Challenges and solutions. In *Proceedings of the 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '19)*. IEEE, 1–6.

[16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805. Retrieved from https:/arxiv.org/abs/1810.04805

[17] Zhang Di, Bing Li, Zengyang Li, and Peng Liang. 2018. A preliminary investigation of self-admitted refactorings in open source software (S). In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering*, Vol. 2018, KSI Research Inc. and Knowledge Systems Institute Graduate School, 165–168.

[18] Malinda Dilhara, Ameya Ketkar, and Danny Dig. 2021. Understanding Software-2.0: A study of machine learning library usage and evolution. *ACM Transactions on Software Engineering and Methodology* 30, 4 (2021), 1–42.

[19] Malinda Dilhara, Ameya Ketkar, Nikhith Sannidhi, and Danny Dig. 2022. Discovering repetitive code changes in Python ML systems. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, New York, NY, 736–748.

[20] Ivan Dokmanic, Reza Parhizkar, Juri Ranieri, and Martin Vetterli. 2015. Euclidean distance matrices: Essential theory, algorithms, and applications. *IEEE Signal Processing Magazine* 32, 6 (2015), 12–30.

[21] Chris Drummond and Robert C. Holte. 2003. C4.5, class imbalance, and cost sensitivity: Why under-sampling beats over-sampling. In *Workshop on Learning from Imbalanced Datasets II*, Vol. 11, ICML, Washington DC, 1–8.

[22] Rudolph Flesch. 1948. A new readability yardstick. *Journal of Applied Psychology* 32, 3 (1948), 221.

[23] Alexandre Babilone Fonseca, David Correa Martins Jr, Zofia Wicik, Marek Postula, and Sérgio Nery Simões. 2021. Addressing classification on highly imbalanced clinical datasets. In *Proceedings of the International Conference on Computational Advances in Bio and Medical Sciences*. Springer, 103–114.

[24] Python Software Foundation. 2023. Retrieved from https://www.python.org/doc/essays/blurb/

[25] Martin Fowler. 2018. *Refactoring*. Addison-Wesley Professional, Berkeley, CA.

[26] Martin Fowler, K. Beck, J. Brant, W. Opdyke, and D. Roberts. 1999. *Refactoring: Improving the Design of Existing Code*. Addison.

[27] Xi Ge, Saurabh Sarkar, Jim Witschey, and Emerson Murphy-Hill. 2017. Refactoring-aware code review. In *Proceedings of the 2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '17)*. IEEE, 71–79.

[28] Migran N. Gevorkyan, Anastasia V. Demidova, Tatiana S. Demidova, and Anton A. Sobolev. 2019. Review and comparative analysis of machine learning libraries for machine learning. *Discrete and Continuous Models and Applied Computational Science* 27, 4 (2019), 305–315.

[29] Awishkar Ghimire, Avinash Kumar Jha, Surendrabikram Thapa, Sushruti Mishra, and Aryan Mani Jha. 2021. Machine learning approach based on hybrid features for detection of phishing URLs. In *Proceedings of the 2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE, 954–959.

[30] Abdelwahab Hamou-Lhadj. 2008. Measuring the complexity of traces using Shannon entropy. In *Proceedings of the 5th International Conference on Information Technology: New Generations (ITNG '08)*. IEEE, 489–494.

[31] David J. Hand. 2012. Assessing the performance of classification methods. *International Statistical Review* 80, 3 (2012), 400–414.
[32] Frank E. Harrell. 2001. *Regression Modeling Strategies: With Applications to Linear Models, Logistic Regression, and Survival Analysis*. Vol. 608, Springer, New York, NY.
[33] Michiel Hazewinkel. 2001. Minimax principle. In *Encyclopedia of Mathematics*. Retrieved from http://encyclopediaofmath.org/index.php?title=Minimax_principle&oldid=44728
[34] Hamid Jafarzadeh, Masoud Mahdianpari, Eric Gill, Fariba Mohammadimanesh, and Saeid Homayouni. 2021. Bagging and boosting ensemble classifiers for classification of multispectral, hyperspectral and PolSAR data: A comparative evaluation. *Remote Sensing* 13, 21 (2021), 4405.
[35] Paul Jansen. 2023. Retrieved from https://www.tiobe.com/tiobe-index/
[36] Jirayus Jiarpakdee, Chakkrit Tantithamthavorn, Akinori Ihara, and Kenichi Matsumoto. 2016. A study of redundant metrics in defect prediction datasets. In *Proceedings of the 2016 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW '16)*. IEEE, 51–52.
[37] Selina Khoirom, Moirangthem Sonia, Borishphia Laikhuram, Jaeson Laishram, and Tekcham Davidson Singh. 2020. Comparative analysis of Python and Java for beginners. *International Research Journal of Engineering and Technology* 7, 8 (2020), 4384–4407.
[38] Miryung Kim, Thomas Zimmermann, and Nachiappan Nagappan. 2014. An empirical study of refactoring challenges and benefits at microsoft. *IEEE Transactions on Software Engineering* 40, 7 (2014), 633–649.
[39] Sotiris Kotsiantis, Dimitris Kanellopoulos, and Panayiotis Pintelas. 2006. Handling imbalanced datasets: A review. *GESTS International Transactions on Computer Science and Engineering* 30, 1 (2006), 25–36.
[40] Marko Leppänen, Samuel Lahtinen, Kati Kuusinen, Simo Mäkinen, Tomi Männistö, Juha Itkonen, Jesse Yli-Huumo, and Timo Lehtonen. 2015. Decision-making framework for refactoring. In *Proceedings of the 2015 IEEE 7th International Workshop on Managing Technical Debt (MTD '15)*. IEEE, 61–68.
[41] Hans Peter Luhn. 1957. A statistical approach to mechanized encoding and searching of literary information. *IBM Journal of Research and Development* 1, 4 (1957), 309–317.
[42] Inderjeet Mani and I. Zhang. 2003. kNN approach to unbalanced data distributions: A case study involving information extraction. In *Proceedings of Workshop on Learning from Imbalanced Datasets*. ICML, 1–7.
[43] Mary L. McHugh. 2012. Interrater reliability: The Kappa statistic. *Biochemia Medica* 22, 3 (2012), 276–282.
[44] Nadia Medeiros, Naghmeh Ivaki, Pedro Costa, and Marco Vieira. 2023. Trustworthiness models to categorize and prioritize code for security improvement. *Journal of Systems and Software* 198 (2023), 111621.
[45] S. Mihajlović, A. Kupusinac, D. Ivetić, and I. Berković. 2020. The use of Python in the field of artificial intelligence. In *Proceedings of the International Conference on Information Technology and Development of Education (ITRO '20)*. University of Novi Sad, Technical Faculty, 124–128.
[46] Tomáš Mikolov, Wen-tau Yih, and Geoffrey Zweig. 2013. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 746–751.
[47] Jeremy Miles. 2005. R-squared, adjusted R-squared. In *Encyclopedia of Statistics in Behavioral Science*, Vol. 1. Retrieved from https://onlinelibrary.wiley.com/doi/abs/10.1002/0470013192.bsa526
[48] Satwik Mishra. 2017. Handling imbalanced data: SMOTE vs. random undersampling. *International Journal of Engineering Research & Technology* 4, 8 (2017), 317–320.
[49] Iman Hemati Moghadam, Mel Ó. Cinnéide, Faezeh Zarepour, and Mohamad Aref Jahanmir. 2021. Refdetect: A multi-language refactoring detection tool based on string alignment. *IEEE Access* 9 (2021), 86698–86727.
[50] Laura Morán-Fernández, Verónica Bólon-Canedo, and Amparo Alonso-Betanzos. 2022. How important is data quality? Best classifiers vs best features. *Neurocomputing* 470 (2022), 365–375.
[51] Thanh H. D. Nguyen, Bram Adams, and Ahmed E. Hassan. 2010. Studying the impact of dependency network measures on software quality. In *Proceedings of the 2010 IEEE International Conference on Software Maintenance*. IEEE, 1–10.
[52] Ehsan Noei, Feng Zhang, and Ying Zou. 2019. Too many user-reviews! What should app developers look at first? *IEEE Transactions on Software Engineering* 47, 2 (2019), 367–378.
[53] Shayan Noei, Heng Li, Stefanos Georgiou, and Ying Zou. 2023. An empirical study of refactoring rhythms and tactics in the software development process. *IEEE Transactions on Software Engineering* 49, 12 (2023), 5103–5119.
[54] Shayan Noei, Heng Li, and Ying Zou. 2024. Replication package. Retrieved from https://github.com/seal-replication-packages/TOSEM2024
[55] Ally S. Nyamawe, Hui Liu, Nan Niu, Qasim Umer, and Zhendong Niu. 2020. Feature requests-based recommendation of software refactorings. *Empirical Software Engineering* 25 (2020), 4315–4347.
[56] Moses Openja, Forough Majidi, Foutse Khomh, Bhagya Chembakottu, and Heng Li. 2022. Studying the practices of deploying machine learning projects on docker. In *Proceedings of the 26th International Conference on Evaluation and Assessment in Software Engineering*. ACM, 190–200.

[57] Julliano Trindade Pintas, Leandro A. F. Fernandes, and Ana Cristina Bicharra Garcia. 2021. Feature selection methods for text classification: A systematic literature review. *Artificial Intelligence Review* 54, 8 (2021), 6149–6200.
[58] Sebastian Raschka, Joshua Patterson, and Corey Nolet. 2020. Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *Information* 11, 4 (2020), 193.
[59] Jacek Ratzinger, Thomas Sigmund, Peter Vorburger, and Harald Gall. 2007. Mining software evolution to predict refactoring. In *Proceedings of the 1st International Symposium on Empirical Software Engineering and Measurement (ESEM '07)*. IEEE, 354–363.
[60] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should i trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1135–1144.
[61] Lior Rokach. 2010. Ensemble-based classifiers. *Artificial Intelligence Review* 33 (2010), 1–39.
[62] Priyadarshni Suresh Sagar, Eman Abdulah AlOmar, Mohamed Wiem Mkaouer, Ali Ouni, and Christian D. Newman. 2021. Comparing commit messages and source code metrics for the prediction refactoring activities. *Algorithms* 14, 10 (2021), 289.
[63] SciTools. 2022. Retrieved from https://support.scitools.com/support/solutions/articles/70000582223-what-metrics-does-understand-have-
[64] Claude Elwood Shannon. 1948. A mathematical theory of communication. *The Bell System Technical Journal* 27, 3 (1948), 379–423.
[65] Mosabbir Khan Shiblu. 2022. *JsDiffer: Refactoring Detection in JavaScript*. Ph. D. Dissertation. Concordia University Montréal, Québec, Canada.
[66] Danilo Silva, Joao Paulo da Silva, Gustavo Santos, Ricardo Terra, and Marco Tulio Valente. 2020. Refdiff 2.0: A multi-language refactoring detection tool. *IEEE Transactions on Software Engineering* 47, 12 (2020), 2786–2802.
[67] Karen Sparck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation* 28, 1 (1972), 11–21.
[68] K. R. Srinath. 2017. Python—The fastest growing programming language. *International Research Journal of Engineering and Technology* 4, 12 (2017), 354–357.
[69] Konstantinos Stroggylos and Diomidis Spinellis. 2007. Refactoring–Does it improve software quality? In *Proceedings of the 5th International Workshop on Software Quality (WoSQ '07: ICSE Workshops 2007)*. IEEE, 10–10.
[70] Reuben Thomas. 2024. Retrieved from https://github.com/rrthomas/enchant
[71] Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig. 2020. RefactoringMiner 2.0. *IEEE Transactions on Software Engineering* 48, 3 (2020), 930–950.
[72] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, 483–494.
[73] Natthida Vatanapakorn, Chitsutha Soomlek, and Pusadee Seresangtakul. 2022. Python code smell detection using machine learning. In *Proceedings of the 2022 26th International Computer Science and Engineering Conference (ICSEC '22)*. IEEE, 128–133.
[74] Zhiyuan Wan, Xin Xia, David Lo, and Gail C. Murphy. 2019. How does machine learning change software development practices? *IEEE Transactions on Software Engineering* 47, 9 (2019), 1857–1871.
[75] Andy Zhou, Kazi Zakia Sultana, and Bharath K. Samanthula. 2021. Investigating the changes in software metrics after vulnerability is fixed. In *Proceedings of the 2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 5658–5663.

Received 9 April 2024; revised 9 August 2024; accepted 15 October 2024